



INSTITUT DE RECHERCHE EN
INFORMATIQUE FONDAMENTALE

Static Typing for Dynamic Languages:

The Case for Set-Theoretic Types and Semantic Subtyping

.....

Giuseppe CASTAGNA
CNRS

Why Static Typing for Dynamic Languages?

Catch bugs early

Type errors are found at compile time, before the code ever runs in production.

Living documentation

Types describe exactly what a function expects and returns, and they are always consistent with the code.

Better tooling

Precise auto-completion, safe refactoring, and IDE navigation all rely on type information.

Safer refactoring

Rename or restructure a codebase with confidence: the type checker catches every inconsistency.

Regulatory compliance

EU and US regulations increasingly **require formally certified software** (CRA, NIST guidelines, ...). A sound type system is a concrete, mechanically verifiable step in that direction. No manual-proofs required.

Socio-Economic and Technical Context

Technical challenges

- Dynamic languages (JavaScript, PHP, Elixir, Python) are widely **used for critical and distributed infrastructures**, precisely because of their flexibility
- They are prone to **hidden vulnerabilities** and are **hard to maintain at scale**
- **Major safety risks** for systems that require formal correctness guarantees

Socio-economic challenges

- Large corporations (Microsoft, Meta, ...) have created **proprietary** typed solutions for JavaScript and PHP, but these remain language-specific and lack formal guarantees
- Full rewrites into statically typed languages require years of work and major resources
- A solution is needed that offers formal guarantees, is **gradual** and **automated**, and remains **open and accessible**

The Challenge: Dynamic Code Does Not Fit Classic Types

Dynamic languages are expressive because they use programming patterns that traditional type systems cannot describe:

1. Non-uniform data structures / branching

A dedicated tag field in a record or a tuple is used as a discriminant to determine the type of the remaining values (tagged unions). **Union types** and **singleton types** are needed.

2. Overloading (ad hoc polymorphism)

The same function behaves differently depending on whether it receives, say, an integer or a string. **Intersection types** are needed.

3. Pattern matching

Branches match specific shapes of data; only certain values can reach each branch. **Negation types** are needed to track what has already been eliminated.

1. Branching and discriminated unions

```
type Status = "pending" | "success" | "error";
type Result = string | number | boolean;
function processStatus(input: Status): Result {
  switch (input) {
    case "pending":
      return "Processing...";           // returns string
    case "success":
      return 200;                       // returns number
    case "error":
      return false;                     // returns boolean
    default:
      return "Unknown status";          // returns string
  }
}
```

1. Branching and discriminated unions

```
type Status = "pending" | "success" | "error";
type Result = string | number | boolean;
function processStatus(input: Status): Result {
  switch (input) {
    case "pending":
      return "Processing...";           // returns string
    case "success":
      return 200;                       // returns number
    case "error":
      return false;                     // returns boolean
    default:
      return "Unknown status";          // returns string
  }
}
```

```
type PaymentMethod =
  | { medium: "credit_card"; cardNumber: string; cvv: string }
  | { medium: "paypal"; email: string }
  | { medium: "transfer"; accountNumber: string; routingNumber: string };
```

2. Overloading: basic use

Aka ad hoc polymorphism: functions that execute different code on arguments of different types

```
function double (x) {  
    return (typeof(x) === "number") ? 2*x : x.concat(x);  
}
```

If applied to an integer returns an integer, if applied to a string returns a string

2. Overloading: basic use

Aka ad hoc polymorphism: functions that execute different code on arguments of different types

```
function double (x) {  
    return (typeof(x) === "number") ? 2*x : x.concat(x);  
}
```

If applied to an integer returns an integer, if applied to a string returns a string

Use set-theoretic types

2. Overloading: basic use

Aka ad hoc polymorphism: functions that execute different code on arguments of different types

```
function double (x) {  
    return (typeof(x) === "number") ? 2*x : x.concat(x);  
}
```

If applied to an integer returns an integer, if applied to a string returns a string

Use set-theoretic types

- Naive solution: union types

$(\text{number} \mid \text{string}) \rightarrow (\text{number} \mid \text{string})$

2. Overloading: basic use

Aka ad hoc polymorphism: functions that execute different code on arguments of different types

```
function double (x) {  
  return (typeof(x) === "number") ? 2*x : x.concat(x);  
}
```

If applied to an integer returns an integer, if applied to a string returns a string

Use set-theoretic types

- Naive solution: union types

$(\text{number} \mid \text{string}) \rightarrow (\text{number} \mid \text{string})$

2. Overloading: basic use

Aka ad hoc polymorphism: functions that execute different code on arguments of different types

```
function double (x) {  
    return (typeof(x) === "number") ? 2*x : x.concat(x);  
}
```

If applied to an integer returns an integer, if applied to a string returns a string

Use set-theoretic types

- Naive solution: union types

$(\text{number} \mid \text{string}) \rightarrow (\text{number} \mid \text{string})$

- Better solution: intersection types

$(\text{number} \rightarrow \text{number}) \ \& \ (\text{string} \rightarrow \text{string})$

2. Overloading: basic use

Aka ad hoc polymorphism: functions that execute different code on arguments of different types

```
function double (x) {  
    return (typeof(x) === "number") ? 2*x : x.concat(x);  
}
```

If applied to an integer returns an integer, if applied to a string returns a string

Use set-theoretic types

- Naive solution: union types

$(\text{number} \mid \text{string}) \rightarrow (\text{number} \mid \text{string})$

- Better solution: intersection types

$(\text{number} \rightarrow \text{number}) \ \& \ (\text{string} \rightarrow \text{string})$

2. Overloading: advanced use

```
function toBoolean(x) {  
  if (x === 0 || x === "" || x == null || Number.isNaN(x)) return false;  
  return true;  
}  
  
function logicalOr(x, y) {  
  if (toBoolean(x)) return x; return y;  
}
```

2. Overloading: advanced use

```
function toBoolean(x) {  
  if (x === 0 || x === "" || x == null || Number.isNaN(x)) return false;  
  return true;  
}  
  
function logicalOr(x, y) {  
  if (toBoolean(x)) return x; return y;  
}
```

Fine grained typing to capture the code semantics

```
type Falsy = 0 | "" | null | undefined | NaN;  
type Truthy =  $\neg$ Falsy;  
toBoolean : (Truthy  $\rightarrow$  true)  $\wedge$  (Falsy  $\rightarrow$  false)  
logicalOR :  $\forall \alpha, \beta. ((\alpha \wedge \text{Truthy} \times \text{Any}) \rightarrow \alpha) \wedge ((\text{Falsy} \times \beta) \rightarrow \beta)$ 
```

3. Precise typing of pattern matching (I)

Consider the following possibly recursive types:

$$T ::= \alpha \mid c \mid \text{Bool} \mid \text{Int} \mid T \times T \mid T \rightarrow T \mid T \vee T \mid T \wedge T \mid \neg T$$

3. Precise typing of pattern matching (I)

Consider the following possibly recursive types:

$$T ::= \alpha \mid c \mid \text{Bool} \mid \text{Int} \mid T \times T \mid T \rightarrow T \mid T \vee T \mid T \wedge T \mid \neg T$$

And let us type the following pattern matching expression:

$$\text{match } e \text{ with } p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2$$

where patterns are defined as follows:

$$p ::= x \mid (p, p) \mid p \mid p \mid p \text{ and } p$$

3. Precise typing of pattern matching (I)

Consider the following possibly recursive types:

$$T ::= \alpha \mid c \mid \text{Bool} \mid \text{Int} \mid T \times T \mid T \rightarrow T \mid T \vee T \mid T \wedge T \mid \neg T$$

And let us type the following pattern matching expression:

`match e with  $p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2$`

where patterns are defined as follows:

$$p ::= x \mid (p, p) \mid p \mid p \mid p \text{ and } p$$

If we interpret types as set of values

$$t = \{v \mid v \text{ is a value of type } t\}$$

3. Precise typing of pattern matching (I)

Consider the following possibly recursive types:

$$T ::= \alpha \mid c \mid \text{Bool} \mid \text{Int} \mid T \times T \mid T \rightarrow T \mid T \vee T \mid T \wedge T \mid \neg T$$

And let us type the following pattern matching expression:

`match e with  $p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2$`

where patterns are defined as follows:

$$p ::= x \mid (p, p) \mid p \mid p \mid p \text{ and } p$$

If we interpret types as set of values

$$t = \{v \mid v \text{ is a value of type } t\}$$

then **the set of all values that match a pattern is a type**

$$\{p\} = \{v \mid v \text{ is a value that matches } p\}$$

(the accepted type of pattern p).

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

```
match e with  $p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2$ 
```

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

```
match e with p1 -> e1 | p2 -> e2
```

Suppose that $e : T$

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with  $p_1 \rightarrow e_1$  |  $p_2 \rightarrow e_2$`

Suppose that $e : T$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with p1 -> e1 | p2 -> e2`

Suppose that $e : T$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;
- To infer the type T_2 of e_2 we need $(T \setminus \{p_1\}) \wedge \{p_2\}$;

($T_1 \setminus T_2$ stands for $T_1 \wedge \neg T_2$)

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with p1 -> e1 | p2 -> e2`

Suppose that $e : T$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;
- To infer the type T_2 of e_2 we need $(T \setminus \{p_1\}) \wedge \{p_2\}$;
- The type of the match expression is $T_1 \vee T_2$.

($T_1 \setminus T_2$ stands for $T_1 \wedge \neg T_2$)

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with p1 -> e1 | p2 -> e2`

Suppose that $e : T$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;
- To infer the type T_2 of e_2 we need $(T \setminus \{p_1\}) \wedge \{p_2\}$;
- The type of the match expression is $T_1 \vee T_2$.
- Pattern matching is exhaustive if $T \leq \{p_1\} \vee \{p_2\}$;

($T_1 \setminus T_2$ stands for $T_1 \wedge \neg T_2$)

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with p1 -> e1 | p2 -> e2`

Suppose that $e : T$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;
- To infer the type T_2 of e_2 we need $(T \setminus \{p_1\}) \wedge \{p_2\}$;
- The type of the match expression is $T_1 \vee T_2$.
- Pattern matching is exhaustive if $T \leq \{p_1\} \vee \{p_2\}$;

($T_1 \setminus T_2$ stands for $T_1 \wedge \neg T_2$)

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with p1 -> e1 | p2 -> e2`

Suppose that $e : T$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;
- To infer the type T_2 of e_2 we need $(T \setminus \{p_1\}) \wedge \{p_2\}$; ($T_1 \setminus T_2$ stands for $T_1 \wedge \neg T_2$)
- The type of the match expression is $T_1 \vee T_2$.
- Pattern matching is exhaustive if $T \leq \{p_1\} \vee \{p_2\}$;

Formally:

[Match]

$$\frac{\Gamma \vdash e : T \quad \Gamma, T \wedge \{p_1\} / p_1 \vdash e_1 : T_1 \quad \Gamma, T \setminus \{p_1\} / p_2 \vdash e_2 : T_2}{\Gamma \vdash \text{match } e \text{ with } p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2 : T_1 \vee T_2} (T \leq \{p_1\} \vee \{p_2\})$$

where T/p is the type for the capture variables in p when it is matched against values in T .

2. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with p1 -> e1 | p2 -> e2`

Suppose that $e : T$ and let us write $T_1 \setminus T_2$ for $T_1 \wedge \neg T_2$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;
- To infer the type T_2 of e_2 we need $(T \setminus \{p_1\}) \wedge \{p_2\}$;
- The type of the match expression is $T_1 \vee T_2$.
- Pattern matching is exhaustive if $T \leq \{p_1\} \vee \{p_2\}$;

Formally:

$$\frac{\begin{array}{c} \text{[Match]} \\ \Gamma \vdash e : T \quad \Gamma, T \wedge \{p_1\} / p_1 \vdash e_1 : T_1 \quad \Gamma, T \setminus \{p_1\} / p_2 \vdash e_2 : T_2 \end{array}}{\Gamma \vdash \text{match } e \text{ with } p_1 \rightarrow e_1 \mid p_1 \rightarrow e_2 : T_1 \vee T_2} \quad (T \leq \{p_1\} \vee \{p_2\})$$

where T/p is the type for the capture variables in p when it is matched against values in T .

Semantic subtyping

Many dynamic languages support union and intersection types: TypeScript, Flow, Hack, Python, Julia, and Ceylon. Negation types are less common: TypeScript (with the 'Exclude' utility type) and Scala (with 'Not' type constraints).

Semantic subtyping

Many dynamic languages support union and intersection types: TypeScript, Flow, Hack, Python, Julia, and Ceylon. Negation types are less common: TypeScript (with the 'Exclude' utility type) and Scala (with 'Not' type constraints).

What differentiates our approach is the use of semantic subtyping: i.e., the use of type semantics to define the subtyping relation for union, intersection, and negation types.

Semantic subtyping

Many dynamic languages support union and intersection types: TypeScript, Flow, Hack, Python, Julia, and Ceylon. Negation types are less common: TypeScript (with the 'Exclude' utility type) and Scala (with 'Not' type constraints).

What differentiates our approach is the use of semantic subtyping: i.e., the use of type semantics to define the subtyping relation for union, intersection, and negation types.

Why not just write down subtyping rules?

Many languages do exactly that — and end up with an incomplete, fragile system: e.g., $\neg(A \vee B)$ not a subtype of $\neg A \wedge \neg B$. With semantic subtyping it comes for **free**

Semantic subtyping

Many dynamic languages support union and intersection types: TypeScript, Flow, Hack, Python, Julia, and Ceylon. Negation types are less common: TypeScript (with the 'Exclude' utility type) and Scala (with 'Not' type constraints).

What differentiates our approach is the use of semantic subtyping: i.e., the use of type semantics to define the subtyping relation for union, intersection, and negation types.

Why not just write down subtyping rules?

Many languages do exactly that — and end up with an incomplete, fragile system: e.g., $\neg(A \vee B)$ not a subtype of $\neg A \wedge \neg B$. With semantic subtyping it comes for **free**

How does it work?

1. Define a set-theoretic interpretation of types as sets of values

Semantic subtyping

Many dynamic languages support union and intersection types: TypeScript, Flow, Hack, Python, Julia, and Ceylon. Negation types are less common: TypeScript (with the 'Exclude' utility type) and Scala (with 'Not' type constraints).

What differentiates our approach is the use of semantic subtyping: i.e., the use of type semantics to define the subtyping relation for union, intersection, and negation types.

Why not just write down subtyping rules?

Many languages do exactly that — and end up with an incomplete, fragile system: e.g., $\neg(A \vee B)$ not a subtype of $\neg A \wedge \neg B$. With semantic subtyping it comes for **free**

How does it work?

1. Define a set-theoretic interpretation of types as sets of values
2. Define union, intersection, and negation types as set-theoretic operations

Semantic subtyping

Many dynamic languages support union and intersection types: TypeScript, Flow, Hack, Python, Julia, and Ceylon. Negation types are less common: TypeScript (with the 'Exclude' utility type) and Scala (with 'Not' type constraints).

What differentiates our approach is the use of semantic subtyping: i.e., the use of type semantics to define the subtyping relation for union, intersection, and negation types.

Why not just write down subtyping rules?

Many languages do exactly that — and end up with an incomplete, fragile system: e.g., $\neg(A \vee B)$ not a subtype of $\neg A \wedge \neg B$. With semantic subtyping it comes for **free**

How does it work?

1. Define a set-theoretic interpretation of types as sets of values
2. Define union, intersection, and negation types as set-theoretic operations
3. Define the subtyping relation as a set-theoretic inclusion relation

Semantic subtyping

Many dynamic languages support union and intersection types: TypeScript, Flow, Hack, Python, Julia, and Ceylon. Negation types are less common: TypeScript (with the 'Exclude' utility type) and Scala (with 'Not' type constraints).

What differentiates our approach is the use of semantic subtyping: i.e., the use of type semantics to define the subtyping relation for union, intersection, and negation types.

Why not just write down subtyping rules?

Many languages do exactly that — and end up with an incomplete, fragile system: e.g., $\neg(A \vee B)$ not a subtype of $\neg A \wedge \neg B$. With semantic subtyping it comes for **free**

How does it work?

1. Define a set-theoretic interpretation of types as sets of values
2. Define union, intersection, and negation types as set-theoretic operations
3. Define the subtyping relation as a set-theoretic inclusion relation
4. Deduce from this how to decide the subtyping relation and compute type operators

Gradual Typing: No Big-Bang Migration

Two non-negotiable requirements for real-world adoption by existing languages:

1. You do not have to type your entire codebase at once.
2. Types do not modify the run-time semantics of existing programs.

Gradual Typing: No Big-Bang Migration

Two non-negotiable requirements for real-world adoption by existing languages:

1. You do not have to type your entire codebase at once.
2. Types do not modify the run-time semantics of existing programs.

The `dynamic()` type

A special type that represents a value whose type is only checked at run time. It acts as an escape hatch that lets untyped and typed code coexist seamlessly.

1. Start with an untyped codebase — no annotations required.
2. Add type annotations incrementally, module by module or function by function.
3. Typed and untyped code interoperate without restriction.
4. As coverage grows, more errors are caught statically.
5. The end goal — a fully typed codebase — gives the same safety guarantees as a fully static system, at your own pace.

Putting the theory to work: Elixir

- Elixir is a dynamic language with a strong emphasis on metaprogramming

Putting the theory to work: Elixir

- Elixir is a dynamic language with a strong emphasis on metaprogramming
- It is a functional language with a syntax similar to Ruby

Putting the theory to work: Elixir

- Elixir is a dynamic language with a strong emphasis on metaprogramming
- It is a functional language with a syntax similar to Ruby
- It is built on top of the Erlang VM, which provides a powerful runtime system for distributed and fault-tolerant applications

Putting the theory to work: Elixir

- Elixir is a dynamic language with a strong emphasis on metaprogramming
- It is a functional language with a syntax similar to Ruby
- It is built on top of the Erlang VM, which provides a powerful runtime system for distributed and fault-tolerant applications
- It has a rich ecosystem of libraries and frameworks and a strong community of nearly 100K developers (with the highest median salaries cf. 2024 Stack Overflow survey)

Putting the theory to work: Elixir

- Elixir is a dynamic language with a strong emphasis on metaprogramming
- It is a functional language with a syntax similar to Ruby
- It is built on top of the Erlang VM, which provides a powerful runtime system for distributed and fault-tolerant applications
- It has a rich ecosystem of libraries and frameworks and a strong community of nearly 100K developers (with the highest median salaries cf. 2024 Stack Overflow survey)
- It is used in production by companies like Discord, Pinterest, PepsiCo, ...

Putting the theory to work: Elixir

- Elixir is a dynamic language with a strong emphasis on metaprogramming
- It is a functional language with a syntax similar to Ruby
- It is built on top of the Erlang VM, which provides a powerful runtime system for distributed and fault-tolerant applications
- It has a rich ecosystem of libraries and frameworks and a strong community of nearly 100K developers (with the highest median salaries cf. 2024 Stack Overflow survey)
- It is used in production by companies like Discord, Pinterest, PepsiCo, ...
- Since 2021 we have been collaborating with José Valim, the creator of Elixir, and the Dashbit team to develop a type system for Elixir. (CIFRE industrial PhD thesis)

Putting the theory to work: Elixir

- Elixir is a dynamic language with a strong emphasis on metaprogramming
- It is a functional language with a syntax similar to Ruby
- It is built on top of the Erlang VM, which provides a powerful runtime system for distributed and fault-tolerant applications
- It has a rich ecosystem of libraries and frameworks and a strong community of nearly 100K developers (with the highest median salaries cf. 2024 Stack Overflow survey)
- It is used in production by companies like Discord, Pinterest, PepsiCo, ...
- Since 2021 we have been collaborating with José Valim, the creator of Elixir, and the Dashbit team to develop a type system for Elixir. (CIFRE industrial PhD thesis)
- The static type system is based on the set-theoretic types we have developed in the past years and being gradually integrated into the Elixir compiler since version 1.17 in 2024 (the current version is 1.19).

Status of the Integration

- Very positive feedback from developers on versions 1.17, 1.18, and 1.19
- No syntactic changes required to Elixir code was particularly appreciated
- Uncovered previously hidden errors without significant overhead
- Found issues in all major projects: Phoenix, Livebook, Postgrex, and Flame. They persisted undetected in production, sometimes for years

Status of the Integration

- Very positive feedback from developers on versions 1.17, 1.18, and 1.19
- No syntactic changes required to Elixir code was particularly appreciated
- Uncovered previously hidden errors without significant overhead
- Found issues in all major projects: Phoenix, Livebook, Postgrex, and Flame. They persisted undetected in production, sometimes for years

Real-world Bug Discoveries

- **Phoenix:** Critical type mismatch in `String.to_atom/1` (deployed on 12,000+ websites)
- **Postgrex:** 15 lines of dead code removed
- **Flame:** 2 dead code paths found via guard analysis
- **Live-view:** 6 lines of dead code removed
- **Remote:** detected a critical type mismatch application in Remote codebase (25K modules) undetected by tests because in an error branch that is rarely executed.

Several Languages use (more or less) Semantic Subtyping:

- **Elixir** — full type system in the compiler since v1.17 (2024), actively used in production
- **Python (ty)** — Astral's high-performance type checker reuses data-structures and optimisations developed for Elixir, and is migrating to a full semantic subtyping implementation.
- **Ballerina** (WSO2) — uses semantic subtyping since its Swan Lake release
- **Luau** (Roblox) — falls back to semantic subtyping when syntactic subtyping is insufficient
- **Erlang** — RPTU (Kaiserlautern) develops *Ethylizer* which reconstructs set-theoretic types for unannotated code by using our semantic subtyping algorithms and techniques; Meta develops *eqWAlizer* to type WhatsApp's Erlang codebase, building on our semantic-subtyping-based gradual typing.
- **R** — U. Karlova (Prague) is developing a type system for R based on semantic subtyping
- **Julia** — core developers cite semantic subtyping as direct inspiration for Julia's type system
- **CDuce** and its spin-offs SSTT and MLsem — the original theoretical proving grounds

Future work

Types for the Elixir Module System:

Currently, only the type `module()` is supported, while we want a finer-grained typing of functions manipulating modules. We are developing solutions applicable to other languages.

Record types comprehensions:

We want to type standard library functions that manipulate records with first-class keys, such as `Map.intersect/2` and `Map.reject/2`.

Types for processes and message passing: Currently, only the type `pid()` is supported. In collaboration with Università degli Studi di Bologna (Italy) and Université d'Evry (France), we are working on typing processes with the set of messages they can receive and, later, we plan to introduce Mailbox types, to ensure liveness properties.

Future work

Tooling: We are developing migration tools, IDEs, error messages, In particular, in collaboration with Ca' Foscari (Venice), we are feeding LLMs with automatic translations of TypeSpecs into Elixir Types together with the type errors they produce to improve the quality of LLM-generated type annotations and error messages.

Application to Ruby: We are collaborating with **NII** and members of the Ruby team to apply our gradual typing solutions to Ruby, which has a syntax similar to Elixir but is object-oriented and has a more complex type system (with classes, modules, ...).

A Proven Approach

1. **Faithful to the semantics.** $T_1 \leq T_2$ means exactly $\llbracket T_1 \rrbracket \subseteq \llbracket T_2 \rrbracket$ — no axioms, no approximations.
2. **Designed for dynamic idioms.** Overloading, pattern matching, and non-uniform branching are all described naturally, without awkward workarounds.
3. **Gradual typing.** Works incrementally. No flag day, no forced full rewrite. Start with nothing and add types where they help.
4. **Proven in production.** Already integrated in Elixir, Python, Ballerina, Erlang. Finding real bugs in real codebases today.
5. **Solid theoretical foundations.** Backed by 15+ years of peer-reviewed work (JACM, POPL, ICFP, OOPSLA).
6. **Supports formal certification.** Soundness is proved, not just tested — directly relevant to regulatory requirements for certified software.

Takeaways

1. Dynamic languages need static typing — for correctness, tooling, and regulatory confidence.

Takeaways

1. Dynamic languages need static typing — for correctness, tooling, and regulatory confidence.
2. Traditional type systems were not built for dynamic idioms. A new model is needed.

Takeaways

1. Dynamic languages need static typing — for correctness, tooling, and regulatory confidence.
2. Traditional type systems were not built for dynamic idioms. A new model is needed.
3. Set-theoretic types with semantic subtyping provide that model: expressive, correct, and well-founded.

Takeaways

1. Dynamic languages need static typing — for correctness, tooling, and regulatory confidence.
2. Traditional type systems were not built for dynamic idioms. A new model is needed.
3. Set-theoretic types with semantic subtyping provide that model: expressive, correct, and well-founded.
4. Gradual adoption means no disruption — start typing today, incrementally.

Takeaways

1. Dynamic languages need static typing — for correctness, tooling, and regulatory confidence.
2. Traditional type systems were not built for dynamic idioms. A new model is needed.
3. Set-theoretic types with semantic subtyping provide that model: expressive, correct, and well-founded.
4. Gradual adoption means no disruption — start typing today, incrementally.
5. Already validated across 8+ languages and real-world production codebases.

Takeaways

1. Dynamic languages need static typing — for correctness, tooling, and regulatory confidence.
2. Traditional type systems were not built for dynamic idioms. A new model is needed.
3. Set-theoretic types with semantic subtyping provide that model: expressive, correct, and well-founded.
4. Gradual adoption means no disruption — start typing today, incrementally.
5. Already validated across 8+ languages and real-world production codebases.
6. **The theory is solid —and still advancing. The practice is proven. The adoption is growing**